Universidad Politécnica Salesiana

Nombres: Jairo Alejandro Ojeda Herrera

Christian Humberto Carrión López

Informe Del Taller Sockets

Implementación de un Worker con latidos UDP, sesión TCP persistente y ejecución asíncrona de tareas

Sistemas Distribuidos

Ing. Julio Ricardo Proaño Orellana

Fecha: 25/10/2026

Periodo: 69



Tabla de contenido

Tabla de ilustraciones 4

1. Introducción 5

2. Objetivos 5

2.1 Objetivo general 5

2.2 Objetivos específicos 5

3. Descripción general de la solución 6

4. Arquitectura y comunicación 6

4.1 Canal TCP 7

4.2 Canal UDP 7

5. Protocolo de mensajes 7

6. Explicación de protocolo.py 8

6.1 Constantes y validación 8

6.2 valid_node_id() 8

6.3 send_json() 8

6.4 recv_json() 8

7. Explicación de worker.py 8

7.1 Parámetros principales 8

7.2 Clase Worker 9

7.3 Logging y eventos 9

7.4 start() 9

7.5 _echo_loop() y _echo_one() 9

7.6 _heartbeat_loop() 9

7.7 run_forever() 9

7.8 _run_job() 9



8. Acciones disponibles del Worker 9

9. Concurrencia y sincronización 10

9.1 tcp_lock 10

9.2 events_lock 10

9.3 stress_lock 10

9.4 ThreadPoolExecutor 10

10. Seguridad y límites implementados 10

11. Flujo de ejecución 11

12. Pruebas y validación 12

12.1 Prueba local 12

12.2 Prueba entre dos computadoras 12

13. Comunicación entre computadoras por Wi-Fi 13

14. Estructura recomendada del proyecto 13

15. Conclusiones 14

16. Recomendaciones 14

Anexo A. Comandos de ejecución 14

Anexo B. Glosario 15

Anexo C. Capturas de pantalla 16

Anexo D. Link GitHub 17

Tabla de ilustraciones

Ilustración 1 Captura del Master 16
Ilustración 2 worker 1 16
Ilustración 3 worker 2 17
Ilustración 4 Historial del worker 2 17

1. Introducción

El presente informe documenta la implementación de un sistema distribuido compuesto por un nodo Worker que se comunica con un nodo Master mediante dos protocolos de red: TCP y UDP. El Worker mantiene una sesión TCP persistente para registrarse, recibir trabajos y devolver resultados, mientras utiliza UDP para enviar periódicamente mensajes de estado o latidos (heartbeat) al Master.

La implementación proporcionada está dividida principalmente en dos archivos: protocolo.py, encargado del intercambio de mensajes JSON sobre TCP, y worker.py, que contiene la lógica del nodo Worker. El diseño también incorpora concurrencia mediante hilos, límites para las pruebas locales, registro de eventos y mecanismos de reconexión.

Es importante aclarar que este documento explica exactamente las funciones que aparecen en el código proporcionado. El código recibido no incluye la implementación del Master, por lo que las funciones del Master se describen únicamente desde el punto de vista de los mensajes que el Worker espera recibir o enviar.

2. Objetivos

2.1 Objetivo general

Implementar y documentar un nodo Worker capaz de comunicarse con un Master mediante TCP y UDP, procesar tareas de manera concurrente y ejecutar pruebas controladas únicamente sobre el propio equipo.

2.2 Objetivos específicos

- Establecer una sesión TCP persistente entre Worker y Master.
- Registrar el Worker mediante un mensaje REGISTER y confirmar el registro mediante REGISTER_ACK.
- Enviar periódicamente información de disponibilidad mediante mensajes UDP PING.
- Recibir trabajos enviados por TCP y procesarlos de forma asíncrona.
- Proporcionar información del sistema, eventos recientes y una comprobación de puerto local.
- Ejecutar una prueba de carga limitada exclusivamente sobre 127.0.0.1.
- Proteger el intercambio de mensajes mediante JSON UTF-8 delimitado por líneas.
- Controlar el acceso concurrente al socket TCP mediante un Lock.

- Registrar eventos tanto en consola como en archivos de log.
- Permitir la reconexión automática cuando se pierde la sesión TCP.

3. Descripción general de la solución

El Worker funciona como un cliente de red. Al iniciar, crea un pequeño servidor TCP de eco únicamente en la interfaz de loopback (127.0.0.1). Este servicio no queda expuesto a la red local. Paralelamente, se inicia un hilo para los heartbeat UDP.

Después, run_forever() intenta conectarse al Master mediante TCP. Una vez conectados, el Worker envía su registro, espera la confirmación y permanece escuchando trabajos. Cada trabajo se ejecuta en un hilo independiente, lo que evita bloquear la recepción de nuevas órdenes.

Componente	**Función principal**
protocolo.py	Serializar y recibir mensajes JSON sobre TCP.
worker.py	Implementar la lógica completa del nodo Worker.
TCP	Registro, recepción de trabajos y devolución de resultados.
UDP	Heartbeat periódico y confirmación mediante ACK.
127.0.0.1	Destino exclusivo para las pruebas locales de eco/carga.
Threading	Permitir heartbeat, recepción y trabajos concurrentes.
Logs	Guardar eventos y advertencias para diagnóstico.

4. Arquitectura y comunicación

La arquitectura utiliza dos canales con objetivos diferentes. TCP se utiliza cuando es necesario mantener una comunicación confiable y ordenada con el Master. UDP se utiliza para los mensajes periódicos de estado.



4.1 Canal TCP

- El Worker se conecta al Master mediante socket.create_connection().
- El puerto TCP utilizado por defecto es 5000.
- El Worker envía REGISTER con su node_id.
- El Master responde REGISTER_ACK.
- El Worker queda esperando mensajes JOB.
- Cada JOB genera una tarea independiente.
- El resultado se devuelve mediante un mensaje RESULT.
- Si la conexión se pierde, el Worker espera aproximadamente 2 segundos y vuelve a intentarlo.

4.2 Canal UDP

- El puerto UDP utilizado por defecto es 5001.
- Cada aproximadamente 2 segundos se genera un heartbeat.
- El mensaje tiene el formato PING:<node_id>|<cpu_usage>|READY.
- El Worker espera un ACK con el formato ACK:<node_id>.
- Si no recibe el ACK, registra una advertencia.

5. Protocolo de mensajes

El archivo protocolo.py establece que los mensajes TCP son objetos JSON codificados en UTF-8 y separados por un salto de línea. Esta estrategia se conoce como JSON Lines o JSONL.

Ejemplo conceptual de registro:

```
{"type":"REGISTER","node_id":"worker1"}
```

Ejemplo conceptual de resultado:

```
{"type":"RESULT","node_id":"worker1","job_id":"123","action":"SYSTEM_INFO","data":{...
}}
```

El límite MAX_MESSAGE_BYTES está establecido en 65536 bytes. Esto evita aceptar tramas TCP excesivamente grandes. Además, recv_json() comprueba que exista el delimitador de línea y que el JSON sea un objeto.

6. Explicación de protocolo.py

6.1 Constantes y validación

MAX_MESSAGE_BYTES = 65536

NODE_ID_PATTERN = re.compile(r"^[A-Za-z0-9_-]{1,32}$")

MAX_MESSAGE_BYTES define el tamaño máximo permitido para una trama. NODE_ID_PATTERN define los caracteres válidos para el identificador del Worker: letras, números, guion y guion bajo, con una longitud de 1 a 32.

6.2 valid_node_id()

Comprueba que el identificador sea una cadena y que cumpla completamente la expresión regular. Esto evita identificadores con caracteres inesperados.

6.3 send_json()

Convierte un diccionario de Python a JSON, agrega un salto de línea, codifica el resultado como UTF-8 y lo envía mediante sendall(). Si se proporciona un Lock, el envío se realiza dentro de la sección protegida.

6.4 recv_json()

Lee una línea desde un stream binario, comprueba el tamaño y el delimitador, decodifica UTF-8 y transforma el JSON a un diccionario de Python. Si la conexión se cerró, devuelve None.

7. Explicación de worker.py

7.1 Parámetros principales

Constante	Valor	Propósito
HEARTBEAT_INTERVAL	2.0 s	Frecuencia aproximada del heartbeat UDP.
MAX_STRESS_SECONDS	10 s	Duración máxima de la prueba local.
LOCAL_CONCURRENCY	2	Máximo de clientes concurrentes de la prueba local.

7.2 Clase Worker

La clase Worker concentra el estado y comportamiento del nodo. Guarda el ID, dirección del Master, puertos, sockets, eventos, locks e hilos de fondo.

7.3 Logging y eventos

_logger() crea la carpeta logs si es necesario y configura dos destinos: consola y archivo. event() añade el evento a una cola limitada a 30 elementos y también lo registra en el logger.

7.4 start()

Crea el servidor de eco local. El bind a 127.0.0.1 es importante: significa que solo puede ser alcanzado desde la propia computadora. También crea el socket UDP e inicia los hilos Echo-local y UDP-heartbeat.

7.5 _echo_loop() y _echo_one()

Estas funciones implementan el eco local. _echo_loop() acepta conexiones y crea un hilo por conexión; _echo_one() recibe hasta 32 bytes y devuelve exactamente los mismos bytes.

7.6 _heartbeat_loop()

Obtiene el porcentaje de CPU mediante psutil cuando está disponible, construye el PING, lo envía al Master y espera el ACK. Si no hay respuesta dentro del tiempo establecido, registra una advertencia.

7.7 run_forever()

Es el ciclo principal de comunicación TCP. Se conecta, registra el Worker, espera trabajos y crea un hilo para cada JOB. Ante una desconexión, limpia la referencia del socket y reintenta después de 2 segundos.

7.8 _run_job()

Interpreta el campo action del mensaje JOB y ejecuta la acción correspondiente. Las acciones implementadas son SYSTEM_INFO, FETCH_LOGS, SCAN_LOCAL_PORT y STRESS_LOCAL. Finalmente, envía un RESULT al Master.

8. Acciones disponibles del Worker

Acción	Qué hace	Resultado
SYSTEM_INFO	Obtiene información del	Diccionario con datos del

	sistema operativo, versión de Python y cantidad de CPU.	sistema.
FETCH_LOGS	Recupera los últimos 10 eventos guardados en memoria.	Lista last_events.
SCAN_LOCAL_PORT	Intenta conectarse al puerto del eco del propio Worker.	Indica que el puerto local está abierto.
STRESS_LOCAL	Ejecuta una prueba acotada contra 127.0.0.1.	Solicitudes exitosas, errores, RPS, latencia promedio y p95.

9. Concurrencia y sincronización

El programa utiliza threading para realizar varias actividades al mismo tiempo. El heartbeat puede ejecutarse mientras el canal TCP permanece escuchando y mientras una tarea está siendo procesada.

9.1 tcp_lock

Un mismo socket TCP puede recibir resultados desde varios hilos. tcp_lock evita que dos hilos escriban simultáneamente y mezclen los bytes de dos mensajes JSON.

9.2 events_lock

Protege la cola de eventos para que diferentes hilos puedan agregar o leer información sin modificarla simultáneamente.

9.3 stress_lock

Impide que se ejecuten dos pruebas STRESS_LOCAL al mismo tiempo en el mismo Worker.

9.4 ThreadPoolExecutor

STRESS_LOCAL utiliza un ThreadPoolExecutor con dos trabajadores. Cada uno realiza conexiones repetidas contra el servicio de eco local, respetando el límite temporal.

10. Seguridad y límites implementados

Aunque el proyecto tiene fines académicos, el código incorpora restricciones importantes para evitar que las pruebas se conviertan en tráfico hacia equipos externos.

- El eco escucha exclusivamente en 127.0.0.1.



- STRESS_LOCAL rechaza cualquier destino distinto de 127.0.0.1.
- La duración de STRESS_LOCAL está limitada entre 1 y 10 segundos.
- La prueba usa como máximo dos hilos concurrentes.
- Las solicitudes se espacian aproximadamente cada 0.25 segundos, limitando el volumen.
- Los mensajes TCP tienen un tamaño máximo de 65536 bytes.
- El node_id tiene formato y longitud controlados.
- Se valida la hora de inicio de STRESS_LOCAL para evitar grandes diferencias de reloj.
- Los sockets tienen timeouts en las operaciones locales y UDP.
- stop() cierra los sockets y espera brevemente a los hilos.

Por estas razones, la función STRESS_LOCAL no es un escaneo de red ni una prueba contra el equipo del compañero: está diseñada específicamente como una simulación controlada sobre loopback.

11. Flujo de ejecución

1. Se ejecuta worker.py con un ID y la dirección IP del Master.
2. Se valida el node_id.
3. Se inicializa el logging y las estructuras internas.
4. Se crea el eco TCP local sobre 127.0.0.1 y un puerto dinámico.
5. Se inicia el hilo de heartbeat UDP.
6. Se intenta crear la conexión TCP con el Master en el puerto 5000.
7. El Worker envía REGISTER.
8. Espera REGISTER_ACK.
9. Comienza a escuchar mensajes JOB.
10. Cuando llega un JOB, se crea un hilo para procesarlo.
11. Se ejecuta la acción solicitada.
12. El Worker devuelve un RESULT.
13. Si se pierde TCP, se intenta reconectar.
14. Al recibir una orden de cierre, se liberan sockets e hilos.

12. Pruebas y validación

Para validar el funcionamiento, se recomienda realizar las pruebas en orden, comenzando por las funciones locales y después verificando la comunicación entre dos computadoras.

12.1 Prueba local

- Iniciar el Worker con un Master local o con la configuración correspondiente.
- Comprobar en consola que aparezca el mensaje de Worker iniciado.
- Verificar que se cree la carpeta logs.
- Comprobar que se genere el archivo de log del Worker.
- Solicitar SYSTEM_INFO y revisar los datos devueltos.
- Solicitar FETCH_LOGS y comprobar que aparecen los eventos recientes.
- Solicitar SCAN_LOCAL_PORT y comprobar que el eco local responde.
- Solicitar STRESS_LOCAL con una duración válida de entre 1 y 10 segundos.

12.2 Prueba entre dos computadoras

- Conectar ambas computadoras a la misma red Wi-Fi o red institucional que permita comunicación entre ellas.
- Identificar la IP privada de la computadora que ejecuta el Master.
- Ejecutar el Worker utilizando esa IP en --master.
- Verificar que el firewall permita el tráfico de los puertos TCP 5000 y UDP 5001 cuando la red lo requiera.
- Comprobar que el Worker reciba REGISTER_ACK.
- Comprobar que el Master reciba los PING y responda con ACK.
- Enviar un JOB y verificar que el Worker devuelva RESULT.

Nota: el hecho de estar conectados por Wi-Fi no impide por sí mismo utilizar TCP/UDP. Lo determinante es que los equipos puedan comunicarse entre sí en la red y que las reglas del firewall o de la infraestructura no bloqueen el tráfico. Algunas redes institucionales aíslan clientes Wi-Fi, en cuyo caso dos equipos pueden tener acceso a Internet pero no poder comunicarse directamente entre ellos.

13. Comunicación entre computadoras por Wi-Fi

Para el escenario trabajado, el Worker debe utilizar como --master la dirección IP de la computadora donde está ejecutándose el Master. No se debe utilizar 127.0.0.1 para conectar al Master remoto, porque 127.0.0.1 siempre representa la propia computadora.

La diferencia fundamental es la siguiente:

Dirección	**Significado**
127.0.0.1	La propia computadora. Solo loopback/local.
IP privada del Master	Dirección que el Worker debe usar para llegar al Master en la red.
0.0.0.0	Normalmente se utiliza al hacer bind para escuchar en todas las interfaces, no como destino del Worker.

El código del Worker mantiene deliberadamente su servicio de eco en 127.0.0.1. Esto permite hacer las pruebas de carga local sin convertir el Worker en un generador de tráfico hacia otros equipos.

14. Estructura recomendada del proyecto

```
proyecto/
├── protocolo.py
├── worker.py
└── logs/
    └── worker1.log
```

La carpeta logs se crea automáticamente cuando se inicia el Worker. No es necesario crearla manualmente. Si se eliminan archivos de pruebas antiguos, no se elimina la funcionalidad principal, siempre que los módulos necesarios del proyecto permanezcan disponibles.

En particular, un archivo de pruebas separado no es utilizado directamente por el código mostrado. Sin embargo, para el informe y para la entrega académica es recomendable conservar pruebas reproducibles o documentar claramente los comandos y resultados obtenidos.

15. Conclusiones

La solución implementa un Worker distribuido que combina TCP y UDP con objetivos diferentes. TCP proporciona un canal persistente para el registro, recepción de tareas y devolución de resultados, mientras UDP permite enviar periódicamente información de disponibilidad mediante heartbeat.

El uso de JSON Lines simplifica el protocolo de aplicación porque cada mensaje puede ser tratado como una unidad delimitada por salto de línea. Además, la aplicación utiliza locks para controlar el acceso concurrente a recursos compartidos y hilos para evitar que una tarea bloquee las demás operaciones del Worker.

Uno de los puntos más importantes del diseño es que las pruebas de carga se encuentran restringidas a 127.0.0.1. Esto permite demostrar concurrencia y medir latencia sin realizar pruebas de tráfico contra máquinas externas. Finalmente, la reconexión automática y el registro de eventos hacen que el Worker sea más tolerante a fallos y más fácil de diagnosticar.

16. Recomendaciones

- Mantener documentadas las IP y puertos utilizados durante las pruebas, sin incluir datos sensibles.
- Realizar las pruebas de red únicamente en redes donde esté permitido el tráfico entre equipos.
- Mantener activado el logging para facilitar la detección de errores.
- Conservar evidencia de las pruebas: capturas, comandos, resultados y logs relevantes.
- No modificar STRESS_LOCAL para apuntar a IPs externas si el objetivo es mantener el diseño seguro del taller.
- Si el proyecto incorpora posteriormente un Master, documentar también su arquitectura y el formato exacto de los JOB y RESULT.
- Antes de la entrega, comprobar que los comandos de ejecución y los nombres de archivos coincidan con la estructura final.

Anexo A. Comandos de ejecución

Ejemplo de ejecución del Worker en la misma computadora:

python worker.py --id worker1 --master 127.0.0.1

Ejemplo conceptual cuando el Master está en otra computadora de la misma red:

python worker.py --id worker1 --master IP_DEL_MASTER

Los puertos por defecto son 5000/TCP y 5001/UDP. También pueden cambiarse mediante --tcp-port y --udp-port.

Anexo B. Glosario

Término	**Significado**
TCP	Protocolo orientado a conexión y confiable, usado aquí para la sesión con el Master.
UDP	Protocolo sin conexión, usado aquí para heartbeat y ACK.
Socket	Punto de comunicación utilizado por el programa para enviar o recibir datos.
Heartbeat	Mensaje periódico que indica que un nodo sigue activo.
ACK	Confirmación de recepción.
JSON	Formato estructurado utilizado para representar los mensajes TCP.
Thread / hilo	Unidad de ejecución concurrente dentro del proceso.
Lock	Mecanismo para impedir acceso simultáneo conflictivo a un recurso compartido.
Loopback	Interfaz local representada por 127.0.0.1.
p95	Percentil 95 de las mediciones de latencia; representa un valor por debajo del cual cae aproximadamente el 95 % de las mediciones.

Anexo C. Capturas de pantalla

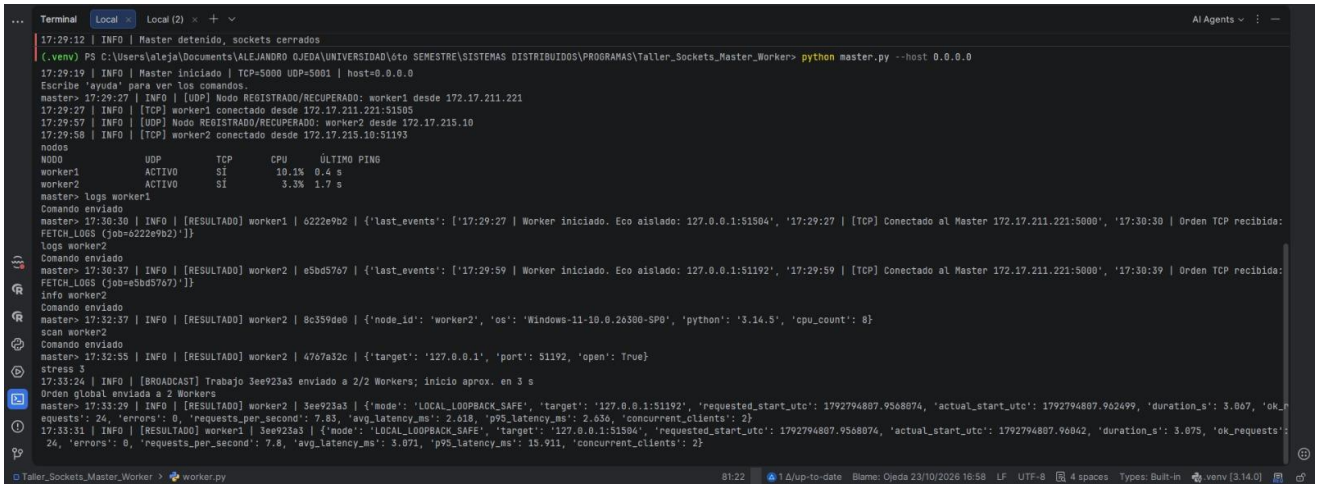


Ilustración 1 Captura del Master

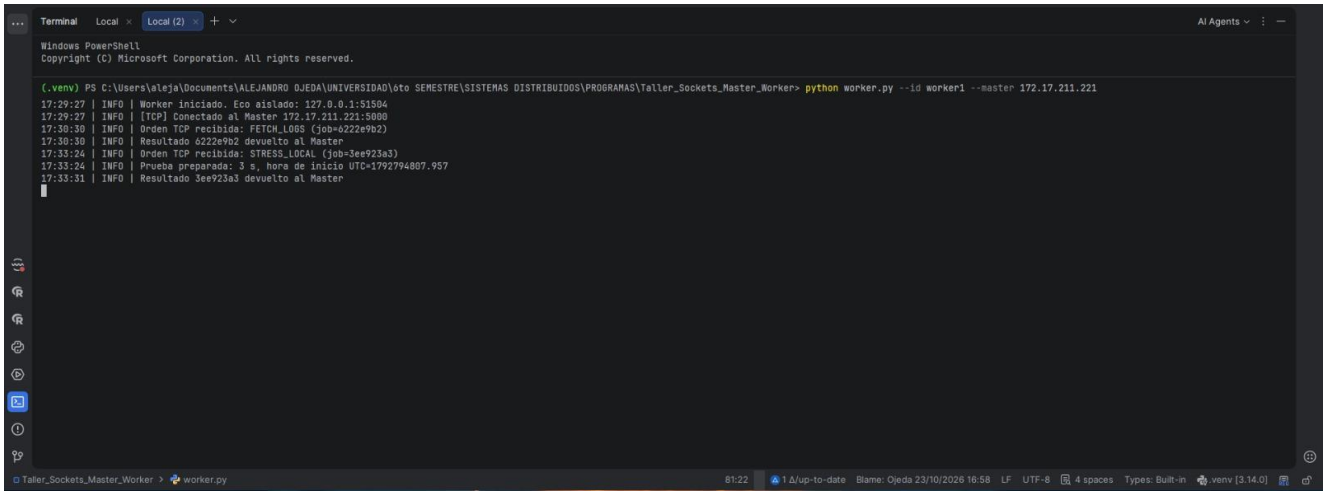


Ilustración 2 worker 1

```
(.venv) PS C:\Users\carri\PycharmProjects\Taller_Sockets_Master_Worker> python worker.py --id worker2 --master 172.17.211.221
17:29:59 | INFO | Worker iniciado. Eco aislado: 127.0.0.1:51192
17:29:59 | INFO | [TCP] Conectado al Master 172.17.211.221:5000
17:30:39 | INFO | Orden TCP recibida: FETCH_LOGS (job=e5bd5767)
17:30:39 | INFO | Resultado e5bd5767 devuelto al Master
17:32:39 | INFO | Orden TCP recibida: SYSTEM_INFO (job=8c359de0)
17:32:39 | INFO | Resultado 8c359de0 devuelto al Master
17:32:57 | INFO | Orden TCP recibida: SCAN_LOCAL_PORT (job=4767a32c)
17:32:57 | INFO | Resultado 4767a32c devuelto al Master
17:33:26 | INFO | Orden TCP recibida: STRESS_LOCAL (job=3ee923a3)
17:33:26 | INFO | Prueba preparada: 3 s, hora de inicio UTC=1792794807.957
17:33:31 | INFO | Resultado 3ee923a3 devuelto al Master
```

Ilustración 3 worker 2

README.md | requirements.txt | worker.py | worker2.log

```
274  17:29:49 | WARNING | Sin ACK UDP (Master no responde o firewall)
275  17:29:51 | WARNING | Sin ACK UDP (Master no responde o firewall)
276  17:29:52 | INFO | Worker detenido; recursos liberados
277  17:29:59 | INFO | Worker iniciado. Eco aislado: 127.0.0.1:51192
278  17:29:59 | INFO | [TCP] Conectado al Master 172.17.211.221:5000
279  17:30:39 | INFO | Orden TCP recibida: FETCH_LOGS (job=e5bd5767)
280  17:30:39 | INFO | Resultado e5bd5767 devuelto al Master
281  17:32:39 | INFO | Orden TCP recibida: SYSTEM_INFO (job=8c359de0)
282  17:32:39 | INFO | Resultado 8c359de0 devuelto al Master
283  17:32:57 | INFO | Orden TCP recibida: SCAN_LOCAL_PORT (job=4767a32c)
284  17:32:57 | INFO | Resultado 4767a32c devuelto al Master
285  17:33:26 | INFO | Orden TCP recibida: STRESS_LOCAL (job=3ee923a3)
286  17:33:26 | INFO | Prueba preparada: 3 s, hora de inicio UTC=1792794807.957
287  17:33:31 | INFO | Resultado 3ee923a3 devuelto al Master
```

Ilustración 4 Historial del worker 2

Anexo D. Link GitHub

Link: https://github.com/Alejandro120325/Taller_Sockets_Master_Worker.git